

MATERIAL EXTRA

Recursos para definição
de tipos



growdev

ENUMS

Enums (ou Enumerações) são uma funcionalidade do TypeScript usada para definir um conjunto de valores nomeados, que podem ser tanto numéricos quanto strings. Eles ajudam a representar de forma clara e organizada opções possíveis, tornando o código mais legível e menos propenso a erros.

Por que usar?

Imagine que você está desenvolvendo uma aplicação onde precisa lidar com direções (cima, baixo, esquerda, direita). Usar strings diretamente ("cima", "baixo", etc.) pode levar a erros de digitação. Enums resolvem esse problema ao garantir valores consistentes e pré-definidos.

Exemplo de Enum numérico:

```
enum Direcao {  
  Cima,  
  Baixo,  
  Esquerda,  
  Direita  
}  
  
const mover = (direcao: Direcao) => {  
  console.log(`Movendo para ${Direcao[direcao]}`);  
};  
  
mover(Direcao.Cima);
```

Exemplo de Enum de string:

```
enum Direcao {  
  Cima,  
  Baixo,  
  Esquerda,  
  Direita  
}  
  
const mover = (direcao: Direcao) => {  
  console.log(`Movendo para ${Direcao[direcao]}`);  
};  
  
mover(Direcao.Cima);
```

ALIAS DE TIPO

Alias de Tipo (Type Alias) é uma forma de criar um nome personalizado para um tipo existente. Ele serve para simplificar a leitura e manutenção do código, principalmente quando se trabalha com tipos complexos.

Por que usar?

Quando um tipo é utilizado várias vezes no código, pode ser cansativo e confuso repetir a definição completa toda vez. Usando um alias, o código se torna mais limpo e fácil de entender.

```
type Ponto = {  
  x: number;  
  y: number;  
};  
  
const desenharPonto = (ponto: Ponto) => {  
  console.log(`Ponto em (${ponto.x}, ${ponto.y})`);  
};  
  
desenharPonto({ x: 10, y: 20 });
```

INTERFACE

Interfaces são uma forma de definir a estrutura de objetos no TypeScript. Elas permitem declarar quais propriedades e métodos um objeto deve ter, facilitando a validação e o autocompletamento no código.

Por que usar?

Interfaces ajudam a garantir que objetos sigam um formato específico, evitando erros causados por propriedades ausentes ou incorretas. Elas são especialmente úteis em projetos maiores, onde várias partes do código precisam usar os mesmos tipos de objetos.

```
interface Usuario {
  id: number;
  nome: string;
  email: string;
}

const exibirUsuario = (usuario: Usuario) => {
  console.log(`Usuário: ${usuario.nome} (${usuario.email})`);
};

exibirUsuario({ id: 1, nome: "João", email: "joao@example.com" });
```

Diferença entre Interface e Alias de Tipo

Embora similares, as interfaces permitem herança (extensão), enquanto os aliases de tipo são mais flexíveis em combinar tipos complexos.

```
interface Animal {
  nome: string;
}

interface Cachorro extends Animal {
  raca: string;
}

const dog: Cachorro = {
  nome: "Rex",
  raca: "Labrador",
};
```

TIPOS DE UNIÃO

Tipos de União permitem que uma variável ou parâmetro aceite mais de um tipo. Isso é útil quando um valor pode variar entre diferentes tipos.

Por que usar?

Muitas vezes, as funções precisam lidar com diferentes tipos de entrada. Os tipos de união garantem que o código manipule corretamente cada caso, evitando erros inesperados.

```
function imprimirId(id: number | string) {  
  if (typeof id === "string") {  
    console.log(`ID: ${id.toUpperCase()}`);  
  } else {  
    console.log(`ID: ${id}`);  
  }  
}  
  
imprimirId(123);  
imprimirId("abc");
```

TIPOS DE INTERSEÇÃO

Tipos de Interseção permitem combinar múltiplos tipos em um único tipo. Uma variável que usa uma interseção deve satisfazer todos os tipos combinados.

Por que usar?

Eles são úteis quando você precisa representar objetos que têm as propriedades de vários tipos, como uma fusão de características.

```
type Endereco = {  
  rua: string;  
  numero: number;  
};  
  
type Pessoa = {  
  nome: string;  
  idade: number;  
};  
  
type PessoaComEndereco = Pessoa & Endereco;  
  
const usuario: PessoaComEndereco = {  
  nome: "Ana",  
  idade: 25,  
  rua: "Rua A",  
  numero: 123,  
};
```

TIPOS LITERAIS

Tipos Literais restringem uma variável a valores específicos. Eles são úteis para criar regras claras sobre quais valores uma variável pode ter.

Por que usar?

Quando você precisa garantir que uma variável só aceite valores predefinidos, os tipos literais ajudam a evitar entradas inválidas.

```
type StatusPedido = "aberto" | "fechado" | "pendente";

function atualizarStatus(status: StatusPedido) {
  console.log(`Status atualizado para: ${status}`);
}

atualizarStatus("aberto");
atualizarStatus("em progresso"); // Erro: não é um valor
permitido
```

TIPOS DE COLEÇÃO

O TypeScript suporta diversos tipos de coleções, como Arrays e Tuplas, com tipagem explícita, ajudando a evitar erros de tipo ao manipular dados.

Arrays

Arrays no TypeScript permitem definir o tipo de seus elementos, garantindo que apenas valores válidos sejam inseridos.

```
const numeros: number[] = [1, 2, 3, 4];

numeros.forEach((numero) => {
  console.log(numero);
});
```

Por que usar Arrays Tipados?

Eles ajudam a evitar a inclusão de elementos de tipos inesperados, tornando o código mais robusto.

Tuplas

Tuplas são coleções com um número fixo de elementos, onde cada elemento pode ter um tipo diferente.

```
type TuplaExemplo = [string, number];

const tupla: TuplaExemplo = ["idade", 30];
console.log(tupla[0]); // "idade"
console.log(tupla[1]); // 30
```

Quando usar Tuplas?

Use tuplas quando precisar representar um conjunto de valores com posições e tipos fixos, como pares chave-valor.

CONCLUSÃO

Esses conceitos básicos do TypeScript são fundamentais para estruturar melhor o código, melhorar a legibilidade e reduzir erros. Com eles, você pode criar aplicações mais robustas e fáceis de manter. Experimente cada exemplo e veja como o TypeScript pode transformar a forma como você escreve código!



BONS ESTUDOS